

СЕМИНАР 5 · ВВЕДЕНИЕ В БАЗЫ ДАННЫХ

# Агрегация в SQL

# Зачем нужна агрегация

---

Часто нужно не выгрузить сырые данные, а рассчитать по ним сводные показатели:

- Сколько рейсов вылетело за прошлый месяц?
- Какова общая сумма продаж или средняя цена перелёта по каждому рейсу?
- Когда вылетел самый первый рейс?

Агрегирующие функции помогают быстро получать ответы на такие вопросы.

# Агрегация данных

**Агрегация данных** — объединение данных из нескольких строк в результирующем наборе на основе группировки.

**Функции агрегации** выполняют расчёты по группам строк и возвращают один результат для каждой группы: сумму, среднее, максимум.

| flight_id | amount   |              | flight_id | max_amount |
|-----------|----------|--------------|-----------|------------|
| 7728      | 14000.00 |              | 7728      | 42100.00   |
| 7728      | 42100.00 | →            | 8725      | 11500.00   |
| 8725      | 10500.00 | MAX (amount) | 9082      | 26600.00   |
| 8725      | 11500.00 |              |           |            |
| 9082      | 8900.00  |              |           |            |
| 9082      | 26600.00 |              |           |            |

# COUNT: количество строк

---

## СИНТАКСИС

```
SELECT COUNT(column1)           AS count_value,  
       COUNT(DISTINCT column1) AS count_distinct_value  
FROM   table;
```

## ПРИМЕР

Найти количество рейсов и количество уникальных аэропортов вылета.

```
SELECT COUNT(flight_id)           AS total_flights,  
       COUNT(DISTINCT departure_airport) AS airports  
FROM   bookings.flights;
```

Результат: total\_flights = 33121, airports = 104

# SUM: сумма значений

---

## СИНТАКСИС

```
SELECT SUM(column1) AS sum_value  
FROM   table;
```

## ПРИМЕР

Найти общую сумму перелётов по рейсу 7728.

```
SELECT SUM(amount) AS total_amount  
FROM   bookings.ticket_flights  
WHERE  flight_id = 7728;
```

Результат: total\_amount = 5802300.00

- Применима к числовым типам данных, в PostgreSQL — ещё и к интервалам

# AVG: среднее значение

---

## СИНТАКСИС

```
SELECT AVG(column1) AS avg_value  
FROM   table;
```

## ПРИМЕР

Найти среднюю цену перелёта.

```
SELECT AVG(amount) AS avg_amount  
FROM   bookings.ticket_flights;
```

Результат: avg\_amount = 19858.912277212195

- Применима к числовым типам данных, в PostgreSQL — ещё и к интервалам

# MIN и MAX: минимум и максимум

## СИНТАКСИС

```
SELECT MIN(column1) AS min_value,  
       MAX(column1) AS max_value  
FROM   table1;
```

## ПРИМЕР

Найти минимальную и максимальную цену перелёта.

```
SELECT MIN(amount) AS min_amount,  
       MAX(amount) AS max_amount  
FROM   bookings.ticket_flights;
```

Результат: min\_amount = 3000.00, max\_amount = 203300.00

- Применимы к числам, датам и текстовым данным

# PERCENTILE\_CONT: перцентиль

## СИНТАКСИС

```
SELECT PERCENTILE_CONT(N) WITHIN GROUP (ORDER BY column)
FROM   table;
```

## ПРИМЕР

Найти медианную цену перелёта.

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median_amount
FROM   bookings.ticket_flights;
```

Результат: median\_amount = 13400

- Вычисляет значение непрерывного перцентиля — полезна, когда нужно понять, как распределены данные

# Агрегирующие функции и NULL

| Функция            | В столбце есть NULL              | Все значения столбца — NULL |
|--------------------|----------------------------------|-----------------------------|
| COUNT (*)          | считает все строки, включая NULL | число строк                 |
| COUNT (column)     | пропускает NULL                  | 0                           |
| SUM, AVG, MIN, MAX | игнорируют NULL                  | NULL                        |
| PERCENTILE_CONT    | игнорирует NULL                  | NULL                        |

ПРИМЕР

Рейсы со статусом «Scheduled»: фактического вылета ещё ни у одного нет.

```
SELECT COUNT(*), COUNT(actual_departure), MAX(actual_departure)
FROM   bookings.flights
WHERE  status = 'Scheduled';
```

Результат: 15383 | 0 | NULL

# Сколько значений заполнено

## ПРИМЕР

Найти количество вылетевших рейсов — тех, у которых заполнен фактический вылет.

### Вариант решения 1

```
SELECT COUNT(*) AS cnt_departed
FROM   bookings.flights
WHERE  actual_departure IS NOT NULL;
```

### Вариант решения 2

```
SELECT COUNT(actual_departure)
       AS cnt_departed
FROM   bookings.flights;
```

## ПРИМЕР

Вывести количество всех рейсов, количество вылетевших и процент вылетевших.

```
SELECT COUNT(*)           AS cnt_all,
       COUNT(actual_departure) AS cnt_departed,
       ROUND(COUNT(actual_departure) * 100.0 / COUNT(*))::int AS departed_prc
FROM   bookings.flights;
```

Результат: cnt\_all = 33121, cnt\_departed = 16773, departed\_prc = 51

# GROUP BY: группировка

## СИНТАКСИС

```
SELECT column1, column2, AGGREGATE_FUNCTION(column3) AS alias
FROM table
GROUP BY column1, column2;
```

## ПРИМЕР

Подсчитать количество перелётов и общую сумму перелётов по каждому классу обслуживания.

```
SELECT fare_conditions,
       COUNT(ticket_no) AS total_tickets,
       SUM(amount) AS total_amount
FROM bookings.ticket_flights
GROUP BY fare_conditions;
```

| fare_conditions | total_tickets | total_amount   |
|-----------------|---------------|----------------|
| Business        | 107642        | 5505179600.00  |
| Comfort         | 17291         | 566116900.00   |
| Economy         | 920793        | 14695684400.00 |

# GROUP BY: что можно писать в SELECT

- Все колонки в SELECT либо указаны в GROUP BY как поля для группировки, либо агрегированы
- Группировать можно по нескольким полям — все они перечисляются в GROUP BY
- GROUP BY без агрегатных функций равносителен DISTINCT

| Запрос                                                                                                                     | Результат                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <pre>SELECT departure_airport, status, COUNT(*)<br/>FROM   bookings.flights<br/>GROUP BY departure_airport, status;</pre>  | 462 строки — по одной на каждую пару «аэропорт — статус»                                                        |
| <pre>SELECT flight_id, fare_conditions, SUM(amount)<br/>FROM   bookings.ticket_flights<br/>GROUP BY fare_conditions;</pre> | ERROR: column "ticket_flights.flight_id" must appear in the GROUP BY clause or be used in an aggregate function |

# Обработка NULL-значений: COALESCE

## СИНТАКСИС

```
SELECT COALESCE(column1, column2 ...) AS not_null_values  
FROM   table;
```

## ПРИМЕР

Для каждого месяца фактического вылета посчитать количество рейсов. Если рейс ещё не вылетел, заменить месяц на 'unknown'.

```
SELECT COALESCE(EXTRACT(MONTH FROM actual_departure)::text, 'unknown') AS departure_month,  
       COUNT(flight_id) AS cnt_flights  
FROM   bookings.flights  
GROUP BY COALESCE(EXTRACT(MONTH FROM actual_departure)::text, 'unknown');
```

Результат: 9 — 9783 рейса, 10 — 6990, unknown — 16348

# HAVING: какие группы оставить

## СИНТАКСИС

```
SELECT
    столбец1,
    АГРЕГАТНАЯ_ФУНКЦИЯ(столбец2) AS алиас
FROM
    таблица
[WHERE условие]
GROUP BY
    столбец1
HAVING
    условие_для_группы;
```

## ПРИМЕР

Найти аэропорты, из которых с 1 октября 2016 года запланировано более 1000 рейсов.

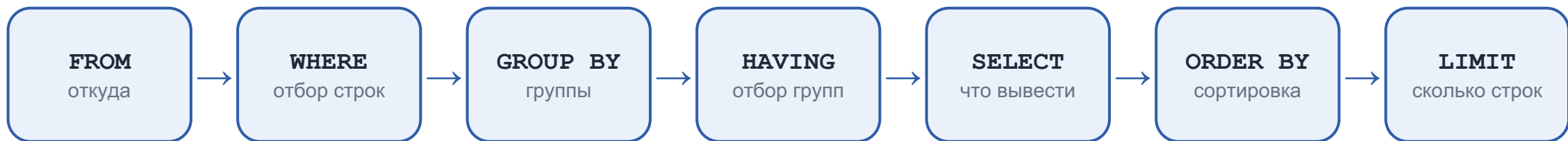
```
SELECT departure_airport,
       COUNT(flight_id) AS total_flights
FROM   bookings.flights
WHERE  scheduled_departure >= '2016-10-01'
GROUP BY departure_airport
HAVING COUNT(flight_id) > 1000;
```

Результат: DME — 2268, SVO — 2101, LED — 1338, VKO — 1212

# WHERE и HAVING: в чём разница

|               | WHERE                                                         | HAVING                                                  |
|---------------|---------------------------------------------------------------|---------------------------------------------------------|
| Что фильтрует | строки до группировки                                         | группы после их формирования                            |
| Когда нужен   | исключить строки, которые не должны участвовать в вычислениях | отфильтровать результаты агрегирующих функций           |
| Что принимает | не может принимать агрегирующие функции                       | агрегирующие функции и поля, по которым шла группировка |

## ПОРЯДОК ВЫПОЛНЕНИЯ НЕ РАВЕН ПОРЯДКУ НАПИСАНИЯ



# Агрегация по части строк: FILTER

## СИНТАКСИС

```
SELECT AGGREGATE_FUNCTION(column1) FILTER (WHERE condition1) AS agg_column1  
FROM table;
```

## ПРИМЕР

Посчитать сумму перелётов бизнес-класса и сумму перелётов эконом-класса.

```
SELECT SUM(amount) FILTER (WHERE fare_conditions = 'Business') AS business_amount,  
       SUM(amount) FILTER (WHERE fare_conditions = 'Economy') AS economy_amount  
FROM bookings.ticket_flights;
```

Результат: business\_amount = 5505179600.00, economy\_amount = 14695684400.00

- Через WHERE эти два значения пришлось бы считать двумя отдельными запросами

# Агрегация по части строк: CASE WHEN

## СИНТАКСИС

```
SELECT
  AGGREGATE_FUNCTION(
    CASE
      WHEN condition1 THEN column1 [ELSE value1]
    END
  ) AS agg_column1
FROM table;
```

## ПРИМЕР

Та же задача: сумма перелётов бизнес-класса и сумма перелётов эконом-класса.

```
SELECT SUM(CASE WHEN fare_conditions = 'Business' THEN amount END) AS business_amount,
       SUM(CASE WHEN fare_conditions = 'Economy' THEN amount END) AS economy_amount
FROM   bookings.ticket_flights;
```

Результат тот же. Внутри CASE WHEN можно собрать не только фильтрацию, но и более сложную логику

# Условная агрегация с GROUP BY: NULL или 0

| Сумма перелётов бизнес-класса для каждого рейса                                                                                                                                                                                                                   | Результат                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <pre>SELECT flight_id,<br/>       SUM(amount) FILTER (WHERE fare_conditions = 'Business')<br/>       AS business_amount<br/>FROM   bookings.ticket_flights<br/>WHERE  flight_id IN (7728, 8725, 9082)<br/>GROUP BY flight_id;</pre>                               | <pre>7728   884100.00<br/>8725   NULL<br/>9082   266000.00</pre> |
| <pre>SELECT flight_id,<br/>       SUM(CASE WHEN fare_conditions = 'Business'<br/>               THEN amount ELSE 0 END)<br/>       AS business_amount<br/>FROM   bookings.ticket_flights<br/>WHERE  flight_id IN (7728, 8725, 9082)<br/>GROUP BY flight_id;</pre> | <pre>7728   884100.00<br/>8725   0<br/>9082   266000.00</pre>    |

У рейса 8725 нет бизнес-класса: все значения под суммой пустые, и SUM возвращает NULL. С ELSE 0 суммируются нули — получается 0.

# Ограничение агрегации и скалярный подзапрос

Нельзя использовать на одном уровне запроса детальные (неагрегированные) поля вместе с агрегированными, если эти детальные поля не указаны в GROUP BY.

```
SELECT flight_id, amount, AVG(amount)
FROM   bookings.ticket_flights;
```

```
ERROR: column "ticket_flights.flight_id" must appear in the GROUP BY clause
       or be used in an aggregate function
```

**Скалярный подзапрос** — вложенный подзапрос, который возвращает ровно одно значение.

Вычисляется один раз (некоррелированный) или по каждой строке (коррелированный). Полезен, если нужно сравнить значение строки с агрегатом.

# Некоррелированный подзапрос в SELECT

## СИНТАКСИС

```
SELECT
    столбец1,
    столбец2,
    (
        SELECT АГРЕГАТНАЯ_ФУНКЦИЯ(столбец3)
        FROM другая_таблица
        WHERE условие
    ) AS алиас
FROM таблица;
```

## ПРИМЕР

Сравнить цену каждого перелёта со средним значением этого показателя по всем данным.

```
SELECT ticket_no, flight_id, amount,
       (SELECT AVG(amount)
        FROM bookings.ticket_flights
       ) AS avg_amount,
       amount - (SELECT AVG(amount)
                  FROM bookings.ticket_flights
                 ) AS diff
FROM   bookings.ticket_flights;
```

Перелёт за 6200.00: avg\_amount = 19858.91..., diff = -13658.91...

Вычисление не зависит от конкретной строки: подзапрос считается один раз.

# Коррелированный подзапрос в SELECT

## СИНТАКСИС

```
SELECT
    столбец1,
    столбец2,
    (
        SELECT АГРЕГАТНАЯ_ФУНКЦИЯ(таблица.столбец3)
        FROM таблица
        WHERE таблица.id_группы
            = внешняя_таблица.id_группы
    ) AS алиас
FROM внешняя_таблица;
```

## ПРИМЕР

Для каждого перелёта рейса 8725 сравнить его цену со средней ценой перелётов того же рейса.

```
SELECT ticket_no, flight_id, amount,
       (SELECT AVG(amount)
        FROM bookings.ticket_flights t1
        WHERE t1.flight_id = t2.flight_id
       ) AS avg_flight_amount
FROM   bookings.ticket_flights t2
WHERE  t2.flight_id = 8725;
```

Результат: 6 строк, avg\_flight\_amount = 10666.67

Вычисляется отдельно для каждой строки внешнего запроса.

# Подведём итоги

---

| Что                                                            | Как пишется                                                                                                  |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Агрегирующие функции                                           | <code>COUNT () , SUM () , MIN () , MAX () , AVG ()</code>                                                    |
| Группировка                                                    | <code>GROUP BY</code>                                                                                        |
| Фильтрация результатов группировки                             | <code>HAVING</code>                                                                                          |
| Агрегация с условиями                                          | <code>агрегация () FILTER (WHERE ...)</code><br><code>агрегация (CASE WHEN ... THEN ... ELSE ... END)</code> |
| Общий агрегат по всем данным или агрегат для конкретной строки | <code>подзапрос в SELECT</code>                                                                              |